Comparative Study of Work-Stealing Schedulers

ABP, Chase-Lev, and Global Queue Scheduling

CS5300: Parallel and Concurrent Programming

Aditya Varun V AI22BTECH11001
Anirudh Saikrishnan CS22BTECH11004
Monish Asawa MA22BTECH11014

***Abstract*—Efficient scheduling of parallel tasks is a central challenge in modern computing systems, particularly for applications that generate work dynamically and exhibit irregular execution patterns. Traditional centralized scheduling approaches often suffer from contention and poor scalability, motivating the need for decentralized alternatives.**

In this report, we present a detailed comparative study of three scheduling strategies: (i) a centralized Single Global Queue, (ii) the bounded Arora-Blumofe-Plaxton (ABP) work-stealing scheduler, and (iii) the dynamic Chase-Lev work-stealing scheduler. Work stealing is a decentralized paradigm in which each worker maintains a local task deque and idle workers dynamically steal tasks from others, enabling efficient load balancing with minimal synchronization overhead.

We evaluate these schedulers using a parallel implementation of the N-Queens problem, a highly irregular backtracking workload that naturally exposes load imbalance. Our analysis focuses on key performance metrics including execution time, speedup, throughput, contention, and load distribution. We further examine scheduler-specific characteristics such as queue overflow in ABP and dynamic resizing in Chase-Lev.

Through this study, we aim to understand the trade-offs between simplicity, scalability, and robustness across different scheduling designs, and provide insights into the suitability of work-stealing techniques for irregular parallel workloads.

I. Introduction

Parallel computing has become essential for achieving high performance in modern applications, ranging from scientific simulations to large-scale data processing. A key challenge in parallel systems is **efficient task scheduling**, particularly when tasks are generated dynamically and exhibit unpredictable execution times.

Many important classes of algorithms naturally produce such workloads. These include recursive divide-and-conquer algorithms, parallel sorting methods, tree-based search procedures, and branch-and-bound techniques. In these settings, the computation often unfolds as a dynamically expanding tree, where different branches may require vastly different amounts of work. As a result, static or naive scheduling approaches can lead to severe **load imbalance**, where some processors remain idle while others are overloaded.

To address this issue, **work stealing** has emerged as a powerful decentralized scheduling strategy. In this approach, each worker thread maintains its own local double-ended queue (deque) of tasks. Workers primarily execute tasks from their own deque, typically in a last-in-first-out (LIFO) manner to preserve cache locality. When a worker becomes idle, it attempts to steal tasks from another worker's deque, usually from the opposite end (first-in-first-out), thereby acquiring larger and potentially more independent tasks.

This design offers several advantages. First, most operations are local and uncontended, reducing synchronization overhead. Second, load balancing is performed dynamically and only when necessary, avoiding unnecessary task movement. Third, theoretical results show that for well-structured computations, work stealing achieves an execution time of the form:

$$T_P \approx \frac{T_1}{P} + O(T_\infty),$$

where T_1 is the total work, T_∞ is the critical path length, and P is the number of processors. This indicates near-optimal parallel performance with low scheduling overhead.

In this report, we study and compare three scheduling approaches:

- A **Single Global Queue**, which serves as a simple centralized baseline.
- The **ABP work-stealing deque**, which uses a bounded array and atomic operations.
- The **Chase-Lev deque**, which extends ABP with dynamic resizing for improved robustness.

Our goal is to analyze how these designs differ in terms of scalability, contention, memory usage, and adaptability to irregular workloads.

II. Problem Definition

The primary objective of this work is to **design, implement, and evaluate different task scheduling strategies** for parallel programs with dynamically generated workloads. Specifically, we aim to compare centralized and decentralized scheduling approaches in terms of their ability to efficiently utilize available processing resources.

A. Scheduling Challenge

In many parallel applications, tasks are not known in advance but are generated recursively during execution. Furthermore, the amount of computation associated with each task can vary significantly. This leads to two major challenges:

- **Unpredictable workload distribution:** Some branches of computation may be significantly deeper or more complex than others.
- **Load imbalance:** Certain workers may finish early while others continue processing long-running tasks, resulting in underutilization of resources.

An effective scheduler must dynamically redistribute work to ensure that all processors remain productive while minimizing overhead.

B. Work-Stealing Framework

To address the above challenges, we adopt the work-stealing paradigm. In this framework:

- Each worker maintains a local deque of tasks.
- The worker pushes and pops tasks from the **bottom** of its deque, typically following LIFO order to improve cache locality.
- Idle workers attempt to steal tasks from the **top** of another worker's deque, usually obtaining older and larger tasks.

This separation of access points reduces contention and allows most operations to proceed without synchronization, while still enabling dynamic load balancing.

C. Schedulers Under Study

We evaluate three different scheduling strategies:

1) **Single Global Queue:**
   - A centralized queue shared by all workers.
   - Requires locking for every task insertion and removal.
   - Simple to implement but suffers from contention and scalability limitations.
2) **ABP Work-Stealing Scheduler:**
   - Uses a fixed-size array-based deque per worker.
   - Employs atomic operations (e.g., compare-and-swap) for synchronization.
   - Offers low contention but may suffer from overflow under heavy workloads.
3) **Chase-Lev Work-Stealing Scheduler:**
   - Uses a dynamically resizable circular array.
   - Retains the benefits of ABP while eliminating fixed capacity limitations.
   - Better suited for highly irregular and unpredictable workloads.

D. Benchmark Problem: N-Queens

To evaluate these schedulers, we use the **N-Queens problem**, a classical combinatorial search problem defined as follows:

- Place N queens on an $N \times N$ chessboard.
- Ensure that no two queens attack each other.
- Compute the total number of valid configurations.

This problem is particularly well-suited for benchmarking scheduling strategies due to:

- Its recursive structure and dynamic task generation.
- Highly irregular search tree with varying branch depths.
- Natural exposure of load imbalance across workers.

Task granularity is controlled using a **split depth**, which determines when new tasks are spawned versus executed sequentially. This allows us to systematically study scheduler behavior under different workload characteristics.

E. Evaluation Metrics

We compare the schedulers using both performance and system-level metrics:

- **Performance metrics:** execution time, speedup, and throughput.
- **Scheduler metrics:** number of successful and failed steals, worker idle time, and load imbalance.
- **System-specific metrics:** queue overflow (ABP) and resize operations (Chase-Lev).

These metrics provide a comprehensive understanding of how each scheduler performs under realistic parallel workloads.

III. Implementation Details

This section describes the concrete implementation of the three scheduling strategies evaluated in this work: the Single Global Queue, the ABP work-stealing deque, and the Chase-Lev work-stealing deque. We also describe the benchmark workload used to evaluate these schedulers.

A. System Overview

All schedulers are implemented using a common backend interface that exposes two primary operations:

- `enqueue(worker_id, task)`: inserts a task into the system.
- `dequeue(worker_id, rng)`: retrieves a task for execution.

Each worker repeatedly invokes `dequeue` to obtain work and executes tasks until termination. Tasks are represented as callable units, enabling flexible execution of recursive workloads such as N-Queens.

B. Task Representation and Execution Model

In our implementation, a task is represented as a heap-allocated callable object:

```
using TaskFunction =
    std::function<void(TaskContext&)>;
using Task = std::shared_ptr<const
    TaskFunction>;
```

Thus, a task does not correspond to a separate class with explicit fields. Instead, the state required for execution is captured inside a C++ lambda. The task receives a `TaskContext`, which provides access to the executing worker and allows the task to spawn additional tasks.

For the N-Queens benchmark, each task represents a partial board configuration. The captured state consists of the current row and three bitmasks: occupied columns, occupied left diagonals, and occupied right diagonals. A task is created as follows:

```
return make_task([this, row, columns,
                  left_diagonals,
    right_diagonals]
                 (TaskContext& context) {
    run_node(context, row, columns,
             left_diagonals, right_diagonals);
});
```

The executor maintains an atomic counter called `outstanding_`, which tracks the number of tasks that have been created but not yet completed. When a task spawns another task, this counter is incremented. When a task finishes execution, the counter is decremented. Execution terminates when the number of outstanding tasks becomes zero.

This design separates task creation from scheduling policy. The same task abstraction is used by all three schedulers,

while the backend determines where the task is placed and how workers retrieve it.

C. Single Global Queue

The global queue scheduler serves as a centralized baseline. All tasks are stored in a shared FIFO queue protected by a mutex.

Design details:

- A single `std::deque<Task>` is used to store tasks.
- A `std::mutex` ensures mutual exclusion for all queue operations.
- `enqueue()` inserts tasks at the back of the queue.
- `dequeue()` removes tasks from the front of the queue.

Key characteristics:

- Every enqueue and dequeue operation requires locking.
- This leads to high contention under increasing parallelism.
- Task distribution is naturally balanced since all workers access the same queue.

Observation from implementation: The simplicity of the design comes at the cost of scalability, as the mutex becomes a bottleneck when multiple workers attempt to access the queue concurrently.

D. ABP Work-Stealing Deque

The ABP scheduler implements a bounded, array-based work-stealing deque per worker. Each worker operates primarily on its own deque, reducing contention.

Data structure:

- A fixed-size array stores tasks.
- Two indices are maintained:
  - `bottom`: modified by the owner thread.
  - `top`: accessed atomically by thieves.

Push operation (owner only):

- Inserts a task at the bottom index.
- Checks for overflow using the condition $bottom - top \geq capacity$.
- Uses release fences to ensure memory visibility.

Pop operation (owner only):

- Removes tasks from the bottom.
- Avoids synchronization in the common case (multiple items).
- Uses a compare-and-swap (CAS) operation only in the last-item race condition with thieves.

Steal operation (thieves):

- Reads from the top index.
- Uses CAS to increment `top` and claim a task.
- Failed CAS attempts are recorded as aborted steals.

Scheduler behavior:

- Workers first attempt local `pop_bottom()`.
- If empty, they randomly select a victim worker and attempt to steal.
- Multiple steal attempts are allowed per polling cycle.

Key characteristics:

- Low contention due to mostly local operations.
- Efficient for moderate workloads.
- Fixed capacity introduces the possibility of overflow.

Implementation-specific metric:

- **Overflow count**: incremented whenever a push fails due to capacity limits.

E. Chase-Lev Work-Stealing Deque

The Chase-Lev scheduler extends the ABP design by removing the fixed capacity limitation through a dynamically resizable circular array.

Data structure:

- A circular array indexed using bitwise masking.
- Capacity is always a power of two.
- The array is managed using `std::shared_ptr` to allow safe resizing.

Push operation (owner only):

- Checks if the array is full.
- If full, allocates a new array with double capacity.
- Copies existing elements from the old array to the new array.
- Publishes the new array using atomic store.

Pop operation (owner only):

- Similar to ABP, with fast-path execution for multiple items.
- Uses CAS only in the last-item race condition.

Steal operation (thieves):

- Reads from the top index.
- Competes using CAS to increment `top`.

Key characteristics:

- Eliminates overflow by dynamically resizing.
- Maintains low contention similar to ABP.
- Slightly higher overhead due to occasional resizing and memory allocation.

Implementation-specific metric:

- **Resize count**: tracks how many times the deque grows during execution.

F. Stealing Strategy

Both ABP and Chase-Lev implementations use randomized victim selection:

- A uniform random worker is selected as the victim.
- Self-stealing is avoided by selecting an alternative worker.
- Multiple steal attempts are made per dequeue call.

Each dequeue attempt records:

- Total steal attempts
- Failed attempts
- Abort cases due to contention

This enables detailed analysis of scheduler efficiency and contention behavior.

G. Benchmark Task: N-Queens

The benchmark used for evaluation is the N-Queens problem, implemented using a recursive backtracking approach.

Task generation:

- Each task represents a partial board configuration.
- At each row, valid placements generate new tasks.

- Tasks are either spawned or executed recursively depending on depth.

Granularity control:

- A **split depth** parameter determines when tasks are spawned.
- For shallow depths, tasks are distributed across workers.
- Beyond the split depth, computation proceeds sequentially.

Execution characteristics:

- Highly irregular search tree.
- Branches vary significantly in size due to pruning.
- Naturally exposes load imbalance.

Why this benchmark is suitable:

- Generates dynamic and unpredictable workloads.
- Stresses both load balancing and scheduling overhead.
- Clearly highlights differences between centralized and work-stealing approaches.

H. Metrics Collection

The implementation collects both performance and scheduler-specific metrics:

- Execution time and throughput
- Number of local vs stolen tasks
- Steal attempts, failures, and aborts
- Overflow count (ABP)
- Resize count (Chase-Lev)

These metrics provide insight into contention, load distribution, and overall scheduler efficiency.

IV. Experimental Setup

To systematically compare the performance and behavior of the three scheduling strategies (Global Queue, ABP Work-Stealing, and Chase-Lev Work-Stealing), we designed a set of controlled experiments across multiple scenarios. Each scenario isolates a specific aspect of the system to evaluate its impact under varying parameter configurations.

A. Common Configuration

Across all experiments, the following baseline settings were used unless explicitly varied:

- Number of repeats: 3, using different random seeds.
- Correctness: All runs verified against known N-Queens solution counts where available.
- ABP capacity (default): 4096.
- Chase-Lev initial log capacity (default): 4, corresponding to an initial capacity of $2^4 = 16$.
- Steal attempt budget (default): 0 in the command-line interface, interpreted internally as one full worker-count-sized victim probing budget per dequeue attempt.

B. Experiment Scenarios

1) 1. Scaling Study: This scenario evaluates how each scheduler scales with increasing parallelism and problem size.

Parameters varied:

- Problem size (n): $\{12, 13, 14, 15\}$
- Number of workers: $\{1, 2, 4, 8, 16\}$
- Split depth: $\{5, 6, 7\}$
- Scheduler: {global, abp, chaselev}

Fixed parameters:

- ABP capacity: 4096
- Chase-Lev log capacity: 4
- Steal attempt budget: 0

This experiment primarily measures scalability in terms of speedup, throughput (tasks per second), and load balance across workers.

2) 2. Task Granularity Study: This scenario analyzes the impact of task granularity on scheduler performance by varying the split depth.

Parameters varied:

- Split depth: $\{2, 3, 4, 5, 6, 7, 8\}$
- Scheduler: {global, abp, chaselev}

Fixed parameters:

- Problem size (n): 13
- Workers: 8
- ABP capacity: 4096
- Chase-Lev log capacity: 4
- Steal attempt budget: 0

This experiment highlights the trade-off between fine-grained parallelism (more tasks, higher overhead) and coarse-grained execution (fewer tasks, potential load imbalance).

3) 3. ABP Capacity Sensitivity: This scenario evaluates the impact of bounded deque capacity in the ABP scheduler.

Parameters varied:

- ABP capacity: $\{4, 8, 16, 32, 64, 128, 256, 512, 1024, 4096\}$

Fixed parameters:

- Scheduler: abp
- Problem size (n): 13
- Workers: 8
- Split depth: 7
- Chase-Lev log capacity: 4
- Steal attempt budget: 0

This experiment specifically measures overflow behavior, including:

- Number of overflow events
- Impact on throughput and scheduling efficiency

4) 4. Chase-Lev Resizing Behavior: This scenario studies the dynamic resizing mechanism of the Chase-Lev deque.

Parameters varied:

- Initial log capacity: $\{1, 2, 3, 4, 5, 6, 7\}$

Fixed parameters:

- Scheduler: chaselev
- Problem size (n): 13
- Workers: 8
- Split depth: 7
- ABP capacity: 4096
- Steal attempt budget: 0

This experiment evaluates:

- Frequency of resizing events
- Overhead of dynamic memory growth
- Effect on work-stealing efficiency

5) 5. Steal Attempt Budget Study: This scenario investigates how limiting the number of steal attempts per polling cycle affects performance.

Parameters varied:

- Steal attempt budget: $\{1, 2, 4, 8, 16, 32\}$
- Scheduler: {abp, chaselev}

Fixed parameters:

- Problem size (n): 13
- Workers: 8
- Split depth: 6
- ABP capacity: 4096
- Chase-Lev log capacity: 4

This experiment measures:

- Trade-off between stealing aggressiveness and overhead
- Failed vs successful steal attempts
- Idle time and load balance

C. Metrics Collected

For each run, we collected the following metrics:

- Execution time and speedup over sequential baseline
- Tasks created, scheduled, and completed
- Throughput (tasks per second)
- Number of successful and failed steal attempts
- Number of steal aborts (contention indicator)
- Idle time across workers
- Load distribution (min, max, mean, standard deviation)
- ABP-specific: overflow count
- Chase-Lev-specific: resize count

All experiments were repeated three times with different random seeds to ensure robustness and reduce variance.

V. Results and Observations

In this section, we present the results obtained from the experimental scenarios described earlier. Each plot highlights a specific aspect of scheduler behavior.

A. Scaling Behavior

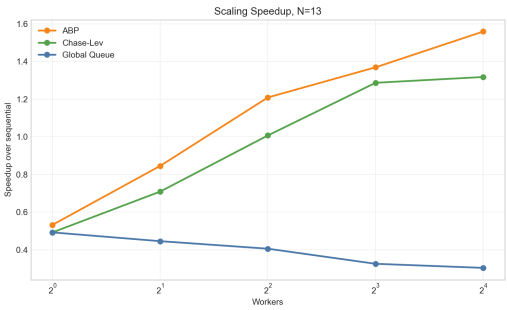

Fig. 1: Speedup vs number of workers for $N = 13$. This plot shows how effectively each scheduler utilizes increasing parallelism relative to the sequential baseline.

Observations:

- Work-stealing schedulers (ABP and Chase-Lev) show consistent reduction in runtime as the number of workers increases, indicating effective parallelization.
- ABP achieves the lowest runtime across all worker counts, suggesting lower overhead in task management compared to Chase-Lev.

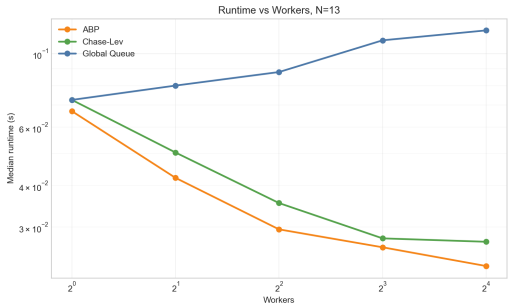

Fig. 2: Runtime vs number of workers (log-log scale). This highlights how execution time decreases (or plateaus) as more workers are added.

- Chase-Lev follows a similar trend but with slightly higher runtime, likely due to resizing and synchronization overheads.
- The global queue scheduler performs poorly as workers increase, with runtime increasing instead of decreasing, indicating contention and lack of scalability.

B. Task Granularity Analysis

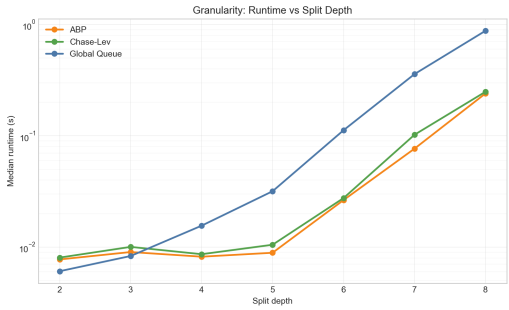

Fig. 3: Runtime vs split depth. This plot illustrates the impact of task granularity on performance, where deeper splitting leads to more parallel tasks.

Fig. 4: Load imbalance (standard deviation of tasks per worker) vs split depth. This shows how evenly work is distributed across workers.

Observations:

- Runtime initially remains low for moderate split depths but increases sharply beyond a threshold, indicating overhead from excessive task creation.
- ABP and Chase-Lev both outperform the global queue across all split depths.

- Load imbalance increases significantly at higher split depths, especially for Chase-Lev, indicating uneven work distribution under fine granularity.
- The global queue maintains relatively lower imbalance but suffers from higher runtime, suggesting that balanced load alone does not guarantee efficiency.

C. ABP Capacity Sensitivity

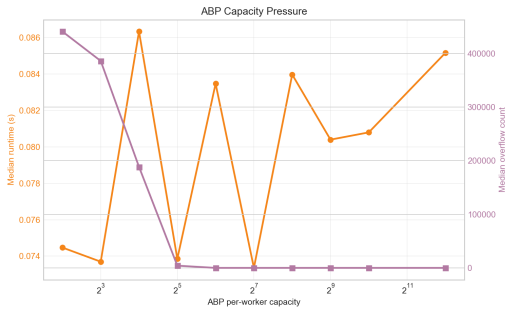

Fig. 5: ABP runtime and overflow count vs deque capacity. The primary axis shows runtime, while the secondary axis shows the number of overflow events.

Observations:

- Very small capacities lead to extremely high overflow counts, indicating frequent fallback to overflow handling mechanisms.
- Overflow count drops sharply as capacity increases, reaching zero beyond a certain threshold.
- Runtime stabilizes once overflows are eliminated, suggesting that overflow handling is a major performance bottleneck.
- Increasing capacity beyond the no-overflow threshold yields diminishing returns in runtime improvement.

D. Chase-Lev Resize Behavior

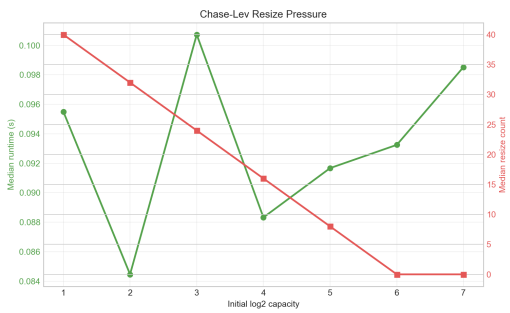

Fig. 6: Chase-Lev runtime and number of resizes vs initial capacity. This plot captures the trade-off between initial capacity and dynamic resizing overhead.

Observations:

- Smaller initial capacities result in a high number of resize operations, indicating frequent dynamic resizing.
- Resize count decreases steadily as initial capacity increases, reaching zero at higher capacities.
- Runtime does not strictly decrease with fewer resizes; instead, it shows a non-monotonic trend, suggesting other overheads (e.g., memory or cache effects).
- There exists an intermediate capacity range where runtime is minimized, balancing allocation cost and resize overhead.

E. Steal Attempt Budget Analysis

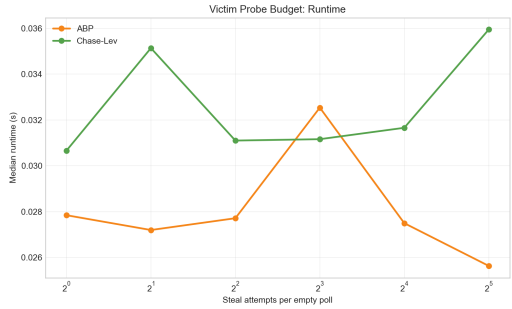

Fig. 7: Runtime vs steal attempt budget. This shows how limiting the number of steal attempts per polling cycle affects overall execution time.

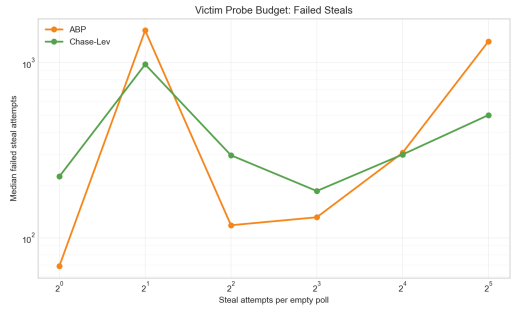

Fig. 8: Failed steal attempts vs steal attempt budget (log scale). This highlights the overhead incurred due to unsuccessful stealing attempts.

Observations:

- Increasing the steal attempt budget increases the number of failed steal attempts significantly, indicating wasted work.
- Runtime tends to improve initially with a small increase in budget but plateaus or degrades beyond a certain point.
- This suggests a trade-off between aggressiveness in stealing and overhead from unsuccessful attempts.
- Optimal performance is achieved at moderate steal budgets rather than extreme values.

F. Summary of Results

The above plots collectively show:

- Work-stealing schedulers scale significantly better than centralized scheduling, with ABP achieving the highest observed speedups (up to $4.24\times$ for $N = 14$) and consistently delivering the fastest runtimes across problem sizes.
- Task granularity must be carefully tuned to balance parallelism and overhead. All schedulers achieved their best performance at low split depths (specifically depth 2), beyond which increasing task granularity led to higher runtimes and overhead.

- Queue capacity and resizing policies have a strong impact on performance. For ABP, overflow effects disappear beyond a capacity of 64, with optimal performance observed around 128. Similarly, Chase-Lev performance depends on the initial capacity, with smaller capacities incurring frequent resizes (e.g., median of 32 resizes at log capacity 2).
- Excessive stealing attempts introduce overhead without proportional gains, as seen from increasing failed steal attempts without corresponding improvements in runtime.

VI. Correctness and Validation

Correctness was validated using known N-Queens solution counts. The sequential solver was first checked against known values, and each scheduler was then required to produce the same number of solutions.

The test suite verifies:

- Sequential N-Queens counts for small problem sizes.
- Correct output for all three schedulers.
- Completion of all created tasks.
- ABP overflow fallback behavior.
- Chase-Lev dynamic resizing behavior.

This ensures that performance comparisons are made only between executions that preserve the same computational result.

VII. Conclusion

This project implemented and compared three task scheduling strategies for irregular parallel workloads: a centralized global queue, a bounded ABP work-stealing deque, and a dynamically resizing Chase-Lev deque. All schedulers were evaluated on the N-Queens benchmark, where tasks are generated dynamically and the search tree exhibits natural load imbalance.

The results show that work-stealing schedulers scale better than the centralized global queue because most operations remain local to each worker and stealing occurs only when workers run out of work. ABP achieved strong performance when the deque capacity was sufficient, but small capacities caused overflow and inline execution. Chase-Lev avoided overflow through dynamic resizing, making it more robust for unpredictable workloads, although resizing introduces additional overhead.

Overall, the study demonstrates that decentralized work stealing is well suited for recursive irregular workloads, but performance depends strongly on task granularity, deque capacity, and stealing aggressiveness.

VIII. References

- Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*, Second Edition. Morgan Kaufmann.
- Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. *Thread Scheduling for Multiprogrammed Multiprocessors*. SPAA, 1998.
- David Chase and Yossi Lev. *Dynamic Circular Work-Stealing Deque*. SPAA, 2005.
- Robert D. Blumofe and Charles E. Leiserson. *Scheduling Multithreaded Computations by Work Stealing*. Journal of the ACM, 1999.